



MANUAL DEL SISTEMA Y ADMINISTRACIÓN DE BASE DE DATOS

VCT STATS

Proyecto: Sistema de Gestión y Estadísticas VCT (Valorant Champions Tour)

DOCENTE: JUAN VELILLA

UNIVERSIDAD LIBRE SECCIONAL

BARRANQUILLA

FACULTAD DE INGENIERÍA

INGENIERÍA DE SISTEMAS

BARRANQUILLA

2026



1. ¿Qué hace el Aplicativo?

El aplicativo VCT Stats es una plataforma integral para la gestión y análisis de torneos del Valorant Champions Tour. El sistema expone una robusta API REST construida con FastAPI en Python, conectada a una base de datos PostgreSQL, diseñada para manejar la carga transaccional y analítica del ecosistema de esports.

Funcionalidades principales:

- **Gestión Administrativa:** Permite el registro, modificación y administración de Torneos, Equipos, Jugadores, Agentes y Mapas.
- **Registro de Partidas (Match History):** Permite registrar cruces entre equipos, establecer resultados, fases del torneo y registrar quién obtuvo la victoria de manera estructurada.
- **Análisis Estadístico Avanzado (Valorant Tracker):**
 - Cálculo de ranking global de jugadores mediante el KDA (Kills, Deaths, Assists).
 - Cálculo del Winrate porcentual de cada equipo en base a los mapas jugados.
 - Tracking del daño y eficacia por Arma usada por los jugadores.
 - Análisis del 'Metajuego', calculando en tiempo real el Pick Rate (tasa de selección) de los diferentes Agentes en las partidas.
- **Autenticación y Seguridad:** Cuenta con endpoints protegidos para login, manejando roles de acceso (Admin y Auditor).

2. Diseño de la Base de Datos

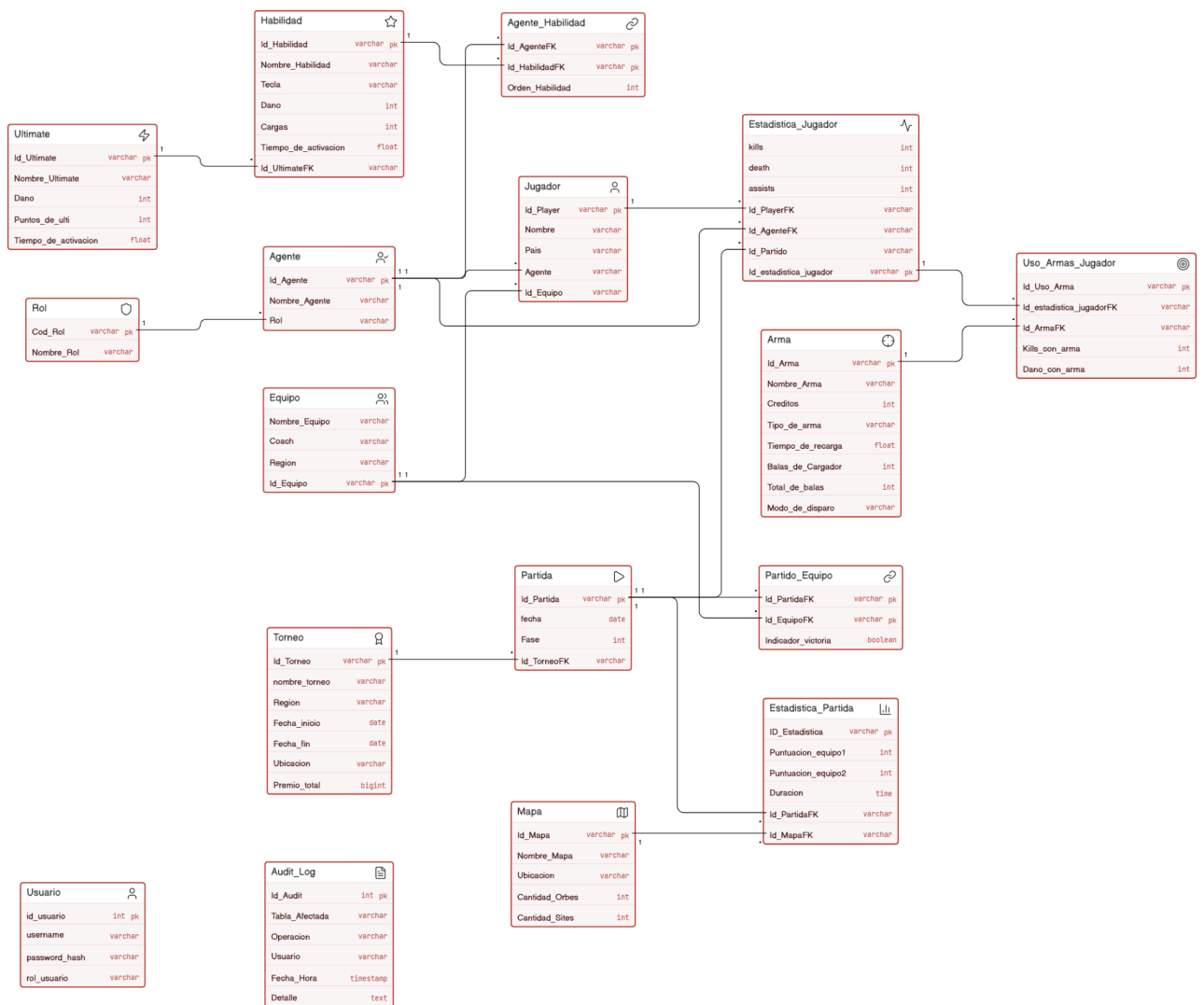


Diagrama Entidad Relación Gestion Estadísticas VCT

El siguiente modelo entidad-relación representa una base de datos diseñada para la gestión y análisis competitivo de un videojuego tipo shooter táctico, inspirado en mecánicas similares a Valorant.

El sistema permite administrar:

- Jugadores
- Equipos
- Agentes
- Habilidades
- Armas
- Partidas
- Torneos
- Estadísticas individuales y colectivas
- Seguridad y auditoría del sistema

La estructura está normalizada y organizada mediante claves primarias (PK) y claves foráneas (FK), permitiendo mantener la integridad referencial entre todas las entidades.



Universidad Libre, Seccional Barranquilla

1. Tabla: Ultimate

Propósito

Almacena la información relacionada con las habilidades definitivas (ultimates) de los agentes.

Campos

Campo	Tipo	Descripción
Id_Ultimate	varchar	Identificador único de la ultimate
Nombre_Ultimate	varchar	Nombre de la habilidad definitiva
Dano	int	Cantidad de daño causado
Puntos_de_ulti	int	Puntos requeridos para activar la ultimate
Tiempo_de_activacion	float	Tiempo que tarda en activarse

Relaciones

- Tiene relación 1 a muchos con Habilidad.
- Una ultimate puede estar asociada a varias habilidades.

2. Tabla: Habilidad

Propósito

Contiene las habilidades normales de cada agente.

Campos

Campo	Tipo	Descripción
Id_Habilidad	varchar	Identificador único
Nombre_Habilidad	varchar	Nombre de la habilidad
Tecla	varchar	Tecla de activación
Dano	int	Daño causado
Cargas	int	Número de usos
Tiempo_de_activacion	float	Tiempo de activación
Id_UltimateFK	varchar	Referencia a la ultimate asociada

Relaciones

- Relación muchos a uno con Ultimate.
- Relación muchos a muchos con Agente mediante Agente_Habilidad.

3. Tabla: Agente

Propósito

Representa los personajes jugables del videojuego.

Campos

Campo	Tipo	Descripción
Id_Agente	varchar	Identificador del agente
Nombre_Agente	varchar	Nombre del agente



Universidad Libre, Seccional Barranquilla

Rol varchar Rol del agente

Relaciones

- Relación muchos a muchos con Habilidad.
- Relación uno a muchos con Jugador.
- Relación uno a muchos con Estadística_Jugador.

4. Tabla: Agente_Habilidad

Propósito

Tabla intermedia para resolver la relación muchos a muchos entre agentes y habilidades.

Campos

Campo	Tipo	Descripción
Id_AgenteFK	varchar	Referencia al agente
Id_HabilidadFK	varchar	Referencia a la habilidad
Orden_Habilidad	int	Orden de aparición o uso

Relaciones

- Muchos registros pertenecen a un agente.
- Muchos registros pertenecen a una habilidad.

Importancia

Permite que:

- Un agente tenga varias habilidades.
- Una habilidad pueda reutilizarse en distintos agentes.

5. Tabla: Rol

Propósito

Define los roles estratégicos de los agentes.

Campos

Campo	Tipo	Descripción
Cod_Rol	varchar	Código del rol
Nombre_Rol	varchar	Nombre del rol

Relaciones

- Relación uno a muchos con Agente.

Ejemplos

- Duelista
- Centinela
- Controlador
- Iniciador

6. Tabla: Jugador

Propósito

Almacena información de los jugadores participantes.

Campos

Campo	Tipo	Descripción
Id_Player	varchar	Identificador del jugador
Nombre	varchar	Nombre del jugador



Universidad Libre, Seccional Barranquilla

Pais	varchar	País de origen
Agente	varchar	Agente principal utilizado
Id_Equipo	varchar	Equipo al que pertenece

Relaciones

- Muchos jugadores pertenecen a un equipo.
- Un jugador puede generar muchas estadísticas.

7. Tabla: Equipo

Propósito

Representa los equipos competitivos.

Campos

Campo	Tipo	Descripción
Nombre_Equipo	varchar	Nombre del equipo
Coach	varchar	Entrenador
Region	varchar	Región competitiva
Id_Equipo	varchar	Identificador único

Relaciones

- Relación uno a muchos con Jugador.
- Relación muchos a muchos con Partida mediante Partido_Equipo.

8. Tabla: Torneo

Propósito

Administra la información de los torneos.

Campos

Campo	Tipo	Descripción
Id_Torneo	varchar	Identificador
nombre_torneo	varchar	Nombre del torneo
Region	varchar	Región
Fecha_inicio	date	Fecha inicial
Fecha_fin	date	Fecha final
Ubicacion	varchar	Lugar del evento
Premio_total	bigint	Premio acumulado

Relaciones

- Un torneo puede contener muchas partidas.

9. Tabla: Partida

Propósito

Representa cada enfrentamiento jugado dentro de un torneo.

Campos



Universidad Libre, Seccional Barranquilla

Campo	Tipo	Descripción
Id_Partida	varchar	Identificador de la partida
fecha	date	Fecha del encuentro
Fase	int	Etapas del torneo
Id_TorneoFK	varchar	Torneo asociado

Relaciones

- Muchas partidas pertenecen a un torneo.
- Relación muchos a muchos con equipos.
- Relación uno a muchos con estadísticas.

10. Tabla: Partido_Equipo

Propósito

Relaciona equipos con partidas específicas.

Campos

Campo	Tipo	Descripción
Id_PartidaFK	varchar	Partida
Id_EquipoFK	varchar	Equipo
Indicador_victoria	boolean	Indica si el equipo ganó

Relaciones

- Permite registrar qué equipos participaron.
- Permite identificar el ganador.

11. Tabla: Mapa

Propósito

Contiene información de los mapas disponibles.

Campos

Campo	Tipo	Descripción
Id_Mapa	varchar	Identificador
Nombre_Mapa	varchar	Nombre del mapa
Ubicacion	varchar	Ambientación
Cantidad_Orbes	int	Número de orbes
Cantidad_Sites	int	Número de zonas objetivo

Relaciones

- Un mapa puede utilizarse en múltiples partidas.

12. Tabla: Estadistica_Partida

Propósito

Almacena estadísticas generales de una partida.

Campos



Universidad Libre, Seccional Barranquilla

Campo	Tipo	Descripción
ID_Estadistica	varchar	Identificador
Puntuacion_equipo1	int	Puntos equipo 1
Puntuacion_equipo2	int	Puntos equipo 2
Duracion	time	Tiempo total
Id_PartidaFK	varchar	Partida asociada
Id_MapasFK	varchar	Mapa utilizado

Relaciones

- Relación muchos a uno con Partida.
- Relación muchos a uno con Mapa.

13. Tabla: Estadistica_Jugador

Propósito

Guarda el rendimiento individual de cada jugador en una partida.

Campos

Campo	Tipo	Descripción
kills	int	Eliminaciones
death	int	Muertes
assists	int	Asistencias
Id_PlayerFK	varchar	Jugador
Id_AgenteFK	varchar	Agente utilizado
Id_Partido	varchar	Partida
Id_estadistica_jugador	varchar	Identificador

Relaciones

- Muchas estadísticas pertenecen a un jugador.
- Muchas estadísticas pertenecen a un agente.
- Muchas estadísticas pertenecen a una partida.

14. Tabla: Arma

Propósito

Contiene la información de las armas del videojuego.

Campos

Campo	Tipo	Descripción
Id_Arma	varchar	Identificador
Nombre_Arma	varchar	Nombre
Creditos	int	Costo



Universidad Libre, Seccional Barranquilla

Tipo_de_arma	varchar	Categoría
Tiempo_de_recarga	float	Tiempo de recarga
Balas_de_Cargador	int	Munición del cargador
Total_de_balas	int	Munición total
Modo_de_disparo	varchar	Tipo de disparo

Relaciones

- Relación muchos a muchos con estadísticas de jugadores mediante Uso_Armas_Jugador.

15. Tabla: Uso_Armas_Jugador

Propósito

Registra el desempeño de un jugador utilizando armas específicas.

Campos

Campo	Tipo	Descripción
Id_Uso_Arma	varchar	Identificador
Id_estadistica_jugadorFK	varchar	Estadística del jugador
Id_ArmaFK	varchar	Arma utilizada
Kills_con_armas	int	Eliminaciones realizadas
Dano_con_armas	int	Daño causado

Relaciones

- Muchos registros pertenecen a una estadística de jugador.
- Muchos registros pertenecen a un arma.

16. Tabla: Usuario

Propósito

Gestiona los usuarios del sistema administrativo.

Campos

Campo	Tipo	Descripción
id_usuario	int	Identificador
username	varchar	Nombre de usuario
password_hash	varchar	Contraseña cifrada
rol_usuario	varchar	Rol dentro del sistema

Importancia

Permite:

- Autenticación
- Control de acceso
- Seguridad del sistema

17. Tabla: Audit_Log

Propósito



Universidad Libre, Seccional Barranquilla

Registra acciones realizadas dentro del sistema.

Campos

Campo	Tipo	Descripción
Id_Audit	int	Identificador
Tabla_Afectada	varchar	Tabla modificada
Operacion	varchar	Tipo de operación
Usuario	varchar	Usuario responsable
Fecha_Hora	timestamp	Fecha y hora
Detalle	text	Descripción detallada

Importancia

Permite:

- Trazabilidad
- Seguridad
- Auditoría de cambios
- Recuperación de incidentes

Relaciones principales del sistema

Relaciones 1:N (Uno a Muchos)

Tabla principal	Tabla relacionada
Ultimate	Habilidad
Rol	Agente
Equipo	Jugador
Torneo	Partida
Partida	Estadistica_Jugador
Mapa	Estadistica_Partida
Partida	Estadistica_Partida

Relaciones N:M (Muchos a Muchos)

Tablas	Tabla intermedia
Agente ↔ Habilidad	Agente_Habilidad
Partida ↔ Equipo	Partido_Equipo
Estadistica_Jugador ↔ Arma	Uso_Armaz_Jugador

Visualizacion dinamica Diagrama BD

<https://app.eraser.io/workspace/3w8KDBRZMcBFeZX3L1QS?origin=share&diagram=JMLSMd8BY5g2Ze>



3. Explicación de Vistas (Views)

Las vistas fueron diseñadas para pre-calcular métricas pesadas y proveer 'Insights' listos para ser consumidos por la API en Python sin saturar la red con JOINS repetitivos.

- **Vista_Resumen_Jugadores**

¿Qué hace?: Genera el 'Ranking' o Leaderboard principal de los jugadores.

Diseño: Une las tablas Jugador, Equipo, Agente y Estadistica_Jugador. Realiza el cálculo matemático en tiempo real del KDA utilizando la fórmula $(Kills + Assists) / Deaths$. Emplea la función `GREATEST(death, 1)` a nivel de motor de BD para evitar caídas del sistema por división sobre cero.

- **Vista_Estadisticas_Equipos**

¿Qué hace?: Muestra el rendimiento general histórico de cada organización (Equipo).

Diseño: Cruza Equipo con Partido_Equipo. Usa conteos condicionales (`COUNT(CASE WHEN...)`) para totalizar partidas jugadas, victorias y derrotas agregadas.

- **vista_detalle_partida**

¿Qué hace?: Compila un historial legible de enfrentamientos.

Diseño: Une Partida, Torneo, Estadistica_Partida y Mapa. Extrae y separa de la tabla intermedia (Partido_Equipo) quién es el equipo 1, quién es el equipo 2 y calcula dinámicamente el nombre del 'ganador' evaluando su indicador de victoria para entregar una fila plana fácil de mostrar en el Frontend.

- **vista_meta_agentes**

¿Qué hace?: Revela el estado del 'Meta' calculando qué agentes son los más seleccionados.

Diseño: A través de CTEs (Common Table Expressions), primero obtiene el total absoluto de partidas jugadas, luego cuenta las elecciones de cada agente en Estadistica_Jugador, cruzando finalmente con la tabla Agente para calcular de manera porcentual exacta el Pick Rate.

4. Explicación de Procedimientos Almacenados (Stored Procedures)

La lógica de negocio transaccional se delegó a PostgreSQL para asegurar los principios ACID.

- **sp_registrar_partida(id, fase, torneo, mapa, e1, e2, s1, s2, duracion)**

¿Qué hace?: Encapsula el flujo atómico para registrar por completo todo el evento de una partida.

Diseño: Realiza las siguientes operaciones en bloque:

1. Crea la instancia principal en la tabla Partida.
2. Genera el cruce relacional del Equipo 1 hacia la partida en Partido_Equipo calculando si el Score 1 > Score 2.
3. Genera el cruce para el Equipo 2 calculando su respectiva victoria/derrota.
4. Crea los metadatos finales en la tabla Estadistica_Partida.



Universidad Libre, Seccional Barranquilla

Beneficio: Al ser ejecutado desde el backend Python (main.py) con una conexión de auto-commit manual, si alguna de las cuatro inserciones falla, el motor realiza Rollback y nada se guarda a medias (evitando datos corruptos).

```
-- 1. Procedimiento para Registrar una Partida Completa
-- Inserta en Partida y Estadistica_Partida en una sola operación
CREATE OR REPLACE PROCEDURE sp_registrar_partida(
    p_id_partida VARCHAR,
    p_fase INTEGER,
    p_id_torneo VARCHAR,
    p_id_mapa VARCHAR,
    p_id_equipo1 VARCHAR,
    p_id_equipo2 VARCHAR,
    p_score1 INTEGER,
    p_score2 INTEGER,
    p_duracion TIME
)
LANGUAGE plpgsql
AS $$
BEGIN
    -- Insertar en la tabla de Partida
    INSERT INTO Partida (Id_Partida, Fase, Id_TorneoFK, fecha)
    VALUES (p_id_partida, p_fase, p_id_torneo, CURRENT_DATE);

    -- Insertar la relacion de los equipos con la partida y quien ganó
    INSERT INTO Partido_Equipo (Id_PartidaFK, Id_EquipoFK, Indicador_victoria)
    VALUES (p_id_partida, p_id_equipo1, p_score1 > p_score2);

    -- Insertar la relacion de los equipos con la partida y quien ganó (Equipo 2)
    INSERT INTO Partido_Equipo (Id_PartidaFK, Id_EquipoFK, Indicador_victoria)
    VALUES (p_id_partida, p_id_equipo2, p_score2 > p_score1);

    -- Insertar en las estadísticas de la partida
    INSERT INTO Estadistica_Partida (ID_Estadistica, Puntuacion_equipo1,
    Puntuacion_equipo2, Duracion, Id_PartidaFK, Id_MapaFK)
    VALUES ('EST' || p_id_partida, p_score1, p_score2, p_duracion, p_id_partida,
    p_id_mapa);

    RAISE NOTICE 'Partida % registrada exitosamente con sus estadísticas.',
    p_id_partida;
EXCEPTION
    WHEN OTHERS THEN
        RAISE EXCEPTION 'Error al registrar la partida: %', SQLERRM;
```



```
END;  
$$;
```

- **sp_actualizar_kda_jugador(id_est, kills, deaths, assists)**

¿Qué hace?: Sobrescribe y actualiza de manera segura y rápida el registro de rendimiento individual de un jugador en una partida dada.

Beneficio: Ejecuta un comando UPDATE sobre los atributos de Estadistica_Jugador buscando por la llave primaria directa de la estadística, reduciendo la fricción y bloqueos en las tablas.

```
-- 2. Procedimiento para Actualizar KDA de Jugador  
CREATE OR REPLACE PROCEDURE sp_actualizar_kda_jugador(  
    p_id_est_jugador VARCHAR,  
    p_kills INTEGER,  
    p_deaths INTEGER,  
    p_assists INTEGER  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    UPDATE Estadistica_Jugador  
    SET kills = p_kills,  
        death = p_deaths,  
        assists = p_assists  
    WHERE Id_estadistica_jugador = p_id_est_jugador;  
  
END;  
$$;
```

5. Explicación de Triggers (Disparadores)

Se han implementado triggers a nivel de la base de datos para ejecutar validaciones y auditoría 'silenciosa', es decir, actúan de forma reactiva a los eventos sin depender del código fuente en Python.

- **trg_auditoria_jugador**

¿Qué hace?: Monitorea de cerca la tabla de Jugadores para detectar cualquier manipulación de la información.

Diseño y Función: Llama a la función PL/pgSQL fn_audit_jugador() activándose inmediatamente AFTER INSERT OR UPDATE OR DELETE sobre la tabla Jugador. Evalúa qué tipo de operación ocurrió (TG_OP) y escribe silenciosamente un log en Audit_Log que captura: el registro afectado, el usuario de la base de datos que realizó el cambio (CURRENT_USER) y el detalle (qué se borró, o cómo se llamaba y cómo se llama ahora usando OLD y NEW).

```
CREATE OR REPLACE FUNCTION fn_audit_jugador()  
RETURNS TRIGGER AS $$
```



```
BEGIN

    IF (TG_OP = 'INSERT') THEN

        INSERT INTO Audit_Log (Tabla_Afectada, Operacion, Usuario, Detalle)

        VALUES ('Jugador', 'INSERT', CURRENT_USER, 'Nuevo jugador: ' || NEW.Nombre);

    ELSIF (TG_OP = 'UPDATE') THEN

        INSERT INTO Audit_Log (Tabla_Afectada, Operacion, Usuario, Detalle)

        VALUES ('Jugador', 'UPDATE', CURRENT_USER, 'Cambio en ID ' || OLD.Id_Player ||
': ' || OLD.Nombre || ' -> ' || NEW.Nombre);

    ELSIF (TG_OP = 'DELETE') THEN

        INSERT INTO Audit_Log (Tabla_Afectada, Operacion, Usuario, Detalle)

        VALUES ('Jugador', 'DELETE', CURRENT_USER, 'Eliminado jugador ID ' ||
OLD.Id_Player);

    END IF;

    RETURN NEW;

END;
```

- **trg_validar_puntuacion**

¿Qué hace?: Mantiene la integridad del modelo evitando errores humanos (como digitar puntuaciones imposibles).

Diseño y Función: Llama a la función fn_validar_puntaje() y se ejecuta BEFORE INSERT OR UPDATE sobre Estadistica_Partida. Intercepta los datos antes de escribirse a disco. Comprueba si NEW.Puntuacion_equipo1 < 0 o NEW.Puntuacion_equipo2 < 0. Si la regla se rompe, levanta una excepción crítica (RAISE EXCEPTION), lo que detiene la transacción entera.

```
-- 2. Función y Trigger de Validación de Puntajes
-- Evita que se inserten puntuaciones negativas

CREATE OR REPLACE FUNCTION fn_validar_puntaje()
RETURNS TRIGGER AS $$
BEGIN

    IF NEW.Puntuacion_equipo1 < 0 OR NEW.Puntuacion_equipo2 < 0 THEN

        RAISE EXCEPTION 'La puntuación de los equipos no puede ser
negativa.';

    END IF;

    RETURN NEW;

END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_puntuacion
BEFORE INSERT OR UPDATE ON Estadistica_Partida
FOR EACH ROW EXECUTE FUNCTION fn_validar_puntaje();
```



Universidad Libre, Seccional Barranquilla

6. Grant Privileges

```
CREATE USER usuario_consulta WITH PASSWORD 'consulta123';
CREATE USER usuario_editor WITH PASSWORD 'editor123';

-- 1. Privilegios básicos para el Consultor
-- Le permitimos conectarse a la base de datos
GRANT CONNECT ON DATABASE vct_stats TO usuario_consulta;
GRANT USAGE ON SCHEMA public TO usuario_consulta;

-- 2. Privilegios básicos para el Editor
GRANT CONNECT ON DATABASE vct_stats TO usuario_editor;
GRANT USAGE ON SCHEMA public TO usuario_editor;

-- 3. Usuario Lewis (Colaborador)
-- Creamos a Lewis con permisos similares al editor para que pueda
trabajar
CREATE USER lewis WITH PASSWORD 'lewis123';
GRANT CONNECT ON DATABASE vct_stats TO lewis;
GRANT USAGE ON SCHEMA public TO lewis;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public
TO lewis;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO lewis;

-- Permisos para que Lewis pueda ver y ejecutar
funciones/procedimientos futuros
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON TABLES TO
lewis;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON FUNCTIONS TO
lewis;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON SEQUENCES TO
lewis;

-- Aseguramos que la tabla exista en la BD
CREATE TABLE IF NOT EXISTS Usuario (
    id_usuario SERIAL PRIMARY KEY,
```



Universidad Libre, Seccional Barranquilla

```
username VARCHAR(50) UNIQUE NOT NULL,  
password_hash VARCHAR(255) NOT NULL,  
rol_usuario VARCHAR(20) DEFAULT 'Auditor' CHECK (rol_usuario IN  
( 'Admin', 'Auditor' ))  
);
```

Se crearon tres usuarios principales con distintos niveles de acceso a la base de datos `vct_stats`:

- **usuario_consulta:**
 - **Permisos:** `CONNECT` a la base de datos y `USAGE` en el esquema `public`.
 - **Propósito:** Acceso básico de consulta (Lectura).
- **usuario_editor:**
 - **Permisos:** `CONNECT` a la base de datos y `USAGE` en el esquema `public`.
 - **Propósito:** Acceso para edición (Inserción, Actualización, Eliminación).
- **lewis (Colaborador):**
 - **Permisos:** Además de `CONNECT` y `USAGE`, se le otorgaron permisos completos (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) en **todas las tablas** del esquema `public`.
 - **Propósito:** Un colaborador con amplios permisos para trabajar. También se configuraron privilegios por defecto para que reciba todos los permisos en futuras tablas, funciones y secuencias.

2. Razón para la creación de la tabla `Usuario`

La tabla `Usuario` fue creada para **gestionar los usuarios del sistema administrativo**. Su propósito principal se centra en la seguridad y el control de acceso del aplicativo VCT Stats.

Importancia: La tabla es fundamental porque permite:

- **Autenticación:** Almacena el `username` y el `password_hash` (contraseña cifrada) para verificar la identidad de quien accede al sistema.
- **Control de Acceso:** Utiliza el campo `rol_usuario` (que puede ser 'Admin' o 'Auditor') para gestionar los roles y restringir las acciones que cada usuario puede realizar dentro del sistema.
- **Seguridad del Sistema:** Garantiza que solo los usuarios registrados con roles definidos puedan interactuar con la lógica administrativa y transaccional del sistema.